

Ex

2026年1月10日 0:15

使用devC来保存未完善的代码，测试代码正确性

1.Dijkstra

- 使用priority_queue（默认大顶堆）来求按消耗燃料最小来进行relax操作

```
// 优先队列：存储(燃料消耗，行，列)，按燃料消耗从小到大排序  
(小顶堆)  
priority_queue<pair<int, pair<int, int>>,  
               vector<pair<int, pair<int, int>>>,  
               greater<pair<int, pair<int, int>>>> pq;
```

2.注意边（图）的存储：两点之间可能有多条边，选权值最小的

edge[x][y]=min(z,edge[x][y]); //覆盖先输入的权

3.注意在初始化的时候不能改变dis[source], SP

4.注意防止溢出问题，检测是否可加/有路径（最短路径中relax操作）

5.图表示

- 邻接表：vector<vector<edge>> e;
 - E[x][y]
- 邻接矩阵：vector<vector<int>> e;
- E.push_back ()

6.特殊处理

- 注意new出来结点，第一个根节点需要特殊处理（先判断）
- 注意特殊list/queue/stack处理第一个元素
 - 修改头指针和尾指针

7.实现一定mod

- Hash
 - 解决冲突时
 - 最后给一定位置赋值的时候，pos一定也不能溢出（详见爸爸去哪儿）
- 循环队列：rear-front+max%max
- 简单字符串循环匹配的移动

8.注意边界，防止越界

- 根节点编号
- 图是1开始还是0，一定!!!
 - 会出现信号终止标志signal terminate（输入没有cin完全也会出现这个）

```
===== 运行崩溃 =====  
程序崩溃原因： SIGNAL_TERMINATE  
退出信号： SIGSEGV  
===== 程序输出 =====
```

- hash函数
- 编号是从0开始还是从1开始

9.利用结构体/类构建节点、边.....

10.有多个输入样例，观察输入的是什麼，输入需要正确

11. 注意i, j, k的使用, 不要混合, 否则循环内部会改变值导致出错

12. Prim算法和Kruskal算法 (学习并查集的实现!!! ——路径压缩)

13.

```
class cmp{
public:
    bool operator()(const node& n1,const node& n2){
        return n1.w<n2.w;
    }
};
```

14. 边列表初始化vector<edge> e(n) 时, 初始创建了 n 个空的边对象, 后续 e.push_back({x,y,z}) 会把输入的边追加到后面, 导致边列表前 n 个元素是无效空边, 排序和遍历都会出错, 应该初始化vector<edge> e或者用e[i]={x,y,z}来赋值存储边

15. STL:

- list双向循环链表
- stack栈
- sort/优先队列
- Lower_bound&upper_bound&binary_search

16. 判断无环

- 拓扑排序
- 广度优先搜索

17. 注意传引用&

18. 判断图能否构成树

- m=n-1
- 是否连通

19. 判断欧拉路径/回路

- 是否连通
- 奇数度顶点个数

20. class不要写成calss了

函数命名不要和C++标识符重名

要注意public

擅于使用class封装

21. 记得初始赋值

```
int find_height(const Node* root,int &size){
    if(root==nullptr)return 0;
    size++;

    int ll=find_height(root->lc,size);
    int rl=find_height(root->rc,size);

    return max(ll,rl)+1;
}

void query(const Node *root, int &size, int &height)
{
    if(root==nullptr)return;
    int ss=0;
    height=find_height(root,ss);
    size=ss;
}
```

22. 注意hash地址冲突都是在hash映射地址的基础上更改的, 不是随更改后的地址更改的!

23. set实现降序排序

- 类似优先队列 `set<int,greater<int>>s;`

- 反向遍历

```
for(set<int>::reverse_iterator it =s.rbegin();it !=s.rend();++it){cout <<*it <<"
";}
```

24.一定要注意判断能不能找到结点，需不需要swap

25.new&delete

26.字母转数字：-'a'

To do list

- ☒ 全排列
- ☒ 全组合
- ☒ KMP
- ☐ 最大生成森林
- ☐ 图的DFS

z13-Compute Active time Interval during DFS

- ☐ 并查集——连通性和亲戚问题

注：路径压缩,最后查询关系也需要find (x)

- ☐ 求交错路径（都与递归深度有关）z10
- ☐ 二分查找
- ☐ 判断树是否平衡/求树高——flag标志 z10
- ☐ 拓扑排序
- ☐ 已知前序中序求后序遍历顺序

一定是传引用!!! 否则拷贝赋值消耗内存且会运行超时 (有时候超时也可考虑一下hash来查找)

- ☐ 小根堆
 - 堆的调整
 - 边界问题，特别是循环和递归终止条件

- ☐ 链表的归并排序mergesort2222
 - 注意极端情况，没进入循环
 - 注意循环终止条件，进入循环时指针或者指针的next非nullptr
 - 注意nullptr
 - 注意边界——空指针，只有一个结点

不那么重要

- ☐ 栈内最小值
- ☐ 求函数的自变量——数学运算符

```
#include<cmath>
```

```
long double solve(long double y)
{
    long double left=0,right=1.0;//得到二分查找区间
    while(exp(right)+log(right)-1<y)//找到右边界
```

☐ 最大值最小化

☐ 中缀转后缀

判断栈是否为空!!!